

StockVisible

Voir la demande que les ruptures cachent, puis décider quoi racheter sous budget — avec des prévisions dont la fiabilité est prouvée.

Date du dossier	27 septembre 2026
Démo en ligne	https://stockvisible.galsentechnologie.com
Secours	https://stockvisible.onrender.com
Dépôt	https://github.com/Mouhinf/stockvisible

En bref

- 20,2 % des heures de 6 h à 22 h sont en rupture dans les données d'entraînement : les ventes observées sous-estiment la demande.
- Trois écrans : Vérifier (la preuve), Comprendre (la demande d'un produit), Acheter (panier sous budget, validé puis exporté).
- Moteur ML choisi sur la validation seule, gelé, puis évalué une seule fois : MAE 3,2 % plus basse que B1 sur le test final, avec un biais négatif affiché à l'écran.
- Données réelles FreshRetailNet-50K (CC BY 4.0), normalisées : aucune unité monétaire, aucune donnée du Sénégal.

Chaque chiffre de ce dossier provient de README.md, logs/final_test_result.json, docs/judging-evidence.md ou AI_USAGE.md (dépôt, branche final-hackathon-polish). Valeurs de vente normalisées, sans unité.

1. Problème et utilisateur

Quand un produit est en rupture, la caisse enregistre zéro vente. Ce zéro n'est pas une absence de clients : les ventes observées sous-estiment la demande, et un gérant qui recommande sur cette base recommande trop peu, ce qui entretient les ruptures.

Dans les données utilisées (500 séries magasin × produit réelles de produits frais), 20,2 % des heures de 6 h à 22 h sont en rupture sur la période d'entraînement.

Utilisateur visé : le gérant ou responsable des achats d'un point de vente de produits frais, qui doit décider chaque jour quoi racheter avec un budget limité. Aucun entretien ni pilote avec un gérant n'a encore eu lieu : ce besoin est une hypothèse de travail, pas un besoin validé sur le terrain.

2. Ce que fait l'application

Le gérant n'a pas besoin de lire le code : l'application lui montre d'abord la preuve de fiabilité (écran Vérifier), puis la demande d'un produit (Comprendre), puis un panier sous budget qu'il valide et exporte (Acheter).

Écran	Ce qu'on y voit
Vérifier (accueil)	Le problème en une phrase, puis la preuve : erreurs du moteur ML et des deux références sur la validation et sur le test final, incertitude, garanties, limites
Comprendre	Pour un produit : ventes heure par heure, disponibilité, ventes observées vs demande estimée par jour, prévisions B0 / B1, abstention ; import d'un fichier contrôlé par le contrat de données
Acheter	Coûts, lots et stocks saisis comme hypothèses ; budget ; panier optimal (demande couverte, manque) ; validation puis export CSV / JSON avec provenance

3. Données

- FreshRetailNet-50K, Dingdong-Inc, licence CC BY 4.0, révision 08c1fab7f9257bc73679d415d65d644165d351d4 (arXiv:2505.16319).
- Sous-ensemble déterministe de 500 séries (seed 42), filtré sans modifier aucune valeur. Ventes normalisées par le fournisseur : aucune unité monétaire.
- Séparation réel / simulé : les données de référence de l'application sont ce benchmark réel. Aucun scénario synthétique en FCFA n'a été construit ; coûts, tailles de lot et stocks sont des hypothèses saisies par l'utilisateur. Les seuls fichiers synthétiques sont ceux des tests, nommés *_SYNTHETIQUE.csv.
- Le fichier « eval » officiel du jeu de données n'est pas versionné et n'a jamais servi à évaluer un modèle.

4. Méthode et résultat

B0 : médiane des ventes au même créneau, ruptures comprises. B1 : même médiane limitée aux heures déclarées disponibles, avec abstention. ML : gradient boosting quantile (scikit-learn) sur des variables strictement passées.

Prévisions horaires glissantes à J+1, heures où le produit était disponible, mêmes heures pour les trois modèles. Sur le test final, ouvert une seule fois, le ML a une erreur 3,2 % plus basse que B1 (5,4 % en validation). Il sous-prévoit (biais négatif, proche de B0) : c'est affiché à l'écran, et c'est pourquoi le panier ne l'utilise pas.

Le test à données masquées est un test de reconstruction / fidélité : il mesure la fidélité de la reconstruction, pas la demande perdue réelle. Ce n'est pas une preuve de récupération de la demande réelle.

Modèle	MAE validation	MAE test final	Biais test final
ML (moteur gelé)	0,0456	0,0492	-0,0355
B1 (médiane, heures disponibles)	0,0482	0,0508	-0,0267
B0 (médiane, ruptures comprises)	0,0475	0,0508	-0,0358

```
{
  "ouvertures_du_test": 1,
  "horodatage_début_utc": "2026-09-26T22:21:13+00:00",
  "horodatage_fin_utc": "2026-09-26T22:23:00+00:00",
  "moteur_gelé": "ML",
  "résultat_moteur_test": {"mae": 0.04915226966771911, "bias": -0.03552760990267266, "couverture": 1.0, "n": 134744},
  "contexte_test": {
    "B0 (glissant)": {"mae": 0.0507987368639791, "bias": -0.03578285489520869, "couverture": 1.0, "n": 134744},
    "B1 (glissant)": {"mae": 0.05077402333313543, "bias": -0.026656808466425222, "couverture": 0.9807611111111111, "n": 134744}
  }
}
```

5. Discipline scientifique

DATA → TRAIN (60 j) → VALIDATION (15 j) → CHOIX DU MOTEUR → GEL → TEST FINAL UNE SEULE FOIS (15 j)

- Découpage chronologique par série, jamais aléatoire.
- Règle de choix écrite avant tout résultat : le ML n'est retenu que s'il bat B1 sur la validation, sinon B1 devient le moteur.
- Anti-fuite testé : canaris, perturbation des jours futurs, et 9 contrôles négatifs (fuites injectées volontairement, que la suite doit détecter).
- Deux regards sur la validation sont déclarés : le premier modèle était dégénéré (il prédisait 0 partout) ; il a été corrigé une fois, puis la configuration a été gelée.

6. Architecture

Fonction	Modules	Rôle
F1 DATA	data.py, validation.py	Sous-ensemble déterministe et vérifié (SHA-256) du jeu de données ; contrat de données ; lecture sûre des fichiers importés (CSV / Parquet, plafonds de taille et de lignes, aucune exécution de contenu)
F2 REVEAL	ui/pages/comprendre.py, ui/charts.py	Ventes horaires, disponibilité déclarée, demande estimée
F3 VERIFY	splits.py, baselines.py, features.py, model.py, evaluation.py, selection.py, final_test.py, uncertainty.py	Découpage chronologique, références B0 / B1, variables strictement passées, modèle ML, sélection sur validation, gel, test final unique, intervalle empirique
F4 DECIDE	allocation.py	Panier optimal sous budget, énumération exhaustive bornée (≤ 3 produits), lots entiers
F5 ACT	exports.py	Validation du panier, export CSV / JSON strictement identique à l'écran

app.py : routeur des trois écrans (st.navigation). Aucun modèle n'est entraîné à l'exécution : l'écran Vérifier lit engine_freeze.json, logs/final_test_result.json et logs/interval_calibration.json.

Interface : Streamlit 1.64, graphiques Plotly, design system dans design-system/MASTER.md. Contrat de données et journal des décisions : docs/data-contract.md.

7. IA responsable

- Pas de LLM ni d'API d'IA externe dans l'application : le moteur est un modèle tabulaire mis en concurrence avec deux règles simples.

- Abstention visible : quand aucune période comparable n'existe, B1 ne prévoit rien et l'écran le dit ; les heures restent vides, jamais remplies par défaut.
- Incertitude mesurée : intervalle 80 % autour du moteur, couverture empirique 79,5 % sur des jours de validation jamais vus en calibration ; « intervalle non calibré » affiché quand le support est insuffisant. La nuit, l'intervalle vaut [0, 0] : calibré mais peu informatif.
- Séparation réel / simulé : données réelles signalées comme telles ; coûts et stocks signalés comme hypothèses ; fichiers de test synthétiques étiquetés ; demande perdue toujours présentée comme estimée. Limites affichées à l'écran.

8. Limites et hypothèses non vérifiées

- Données réelles chinoises (Dingdong), normalisées : aucune unité monétaire, aucun FCFA, aucune donnée du Sénégal.
- Coûts, tailles de lot et stocks sont des hypothèses saisies : le jeu de données n'en contient pas.
- La demande perdue pendant une rupture est estimée, jamais observée. Le test à données masquées mesure la fidélité de la reconstruction, pas la demande perdue réelle.
- Gain du ML modeste (3,2 % sur le test) et biais négatif ; le panier utilise B1.
- Les lignes de la période test ont été lues avant le découpage (contrôle qualité des données) et sont publiques dans le dépôt ; elles n'ont servi à aucune décision de modèle.
- Offre gratuite Render : démarrage à froid observé d'environ 2 minutes après 15 minutes d'inactivité.
- Aucun entretien ni pilote avec un gérant : le besoin utilisateur reste une hypothèse de travail.

9. Critère du jury → preuve

Critère (poids)	Preuve	Test
Problème et valeur (20)	20,2 % des heures 6 h–22 h en rupture ; bandeau du problème calculé sur les données ; ventes observées et demande estimée côte à côte	test_problem_banner_numbers_are_computed_not_typed
Fonctionnement (20)	Parcours import → visualisation → preuve → budget → panier → validation → export, local et production, ordinateur et mobile, zéro erreur console ; export identique à l'écran	e2e/parcours.spec.ts ; test_export_is_strictly_what_the_screen_shows
Usage de l'IA (20)	ML contre B0 / B1, règle écrite avant tout résultat, choix sur la validation seule, configuration gelée (empreintes SHA-256) ; test final ouvert une seule fois	test_select_engine_never_receives_test_data ; test_result_is_a_single_opening_of_the_frozen_engine
Tests et fiabilité (15)	Découpage reproductible ; anti-fuite par canaris et perturbation ; 9 contrôles négatifs ; répétition générale du test final ; CI GitHub Actions	test_anti_leak_fails_on_injected_leak ; test_dress_rehearsal_on_validation_reproduces_frozen_metrics
Expérience et démo (15)	Trois écrans Vérifier → Comprendre → Acheter ; au plus trois indicateurs par écran ; contrastes WCAG AA calculés	test_at_most_three_metrics_per_screen ; test_every_text_color_meets_wcag_aa
IA responsable et données (10)	Abstention visible ; couverture de l'intervalle 79,5 % ; données réelles CC BY 4.0, révision figée ; hypothèses et fichiers synthétiques étiquetés	test_synthetic_series_without_enough_history_triggers_abstention ; test_real_report_uses_validation_only_and_is_consistent

10. Installation, tests et déploiement

Installation (Python 3.12 requis)

```
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/streamlit run app.py # http://localhost:8501
```

```
.venv/bin/ruff check .
.venv/bin/pytest -q # suite complète (519 tests collectés)
.venv/bin/pytest -q -m "not touches_test_period" # sans lire la période test réservée
npm ci && npx playwright test # parcours E2E, ordinateur + mobile
E2E_BASE_URL=https://stockvisible.onrender.com npx playwright test # même parcours en production
```

Sur un clone neuf, 2 tests sont sautés car ils demandent des fichiers de données non versionnés (eval officiel, cache de téléchargement), avec une raison explicite. CI GitHub Actions : ruff, pytest, pip-audit (informatif). Instance Render gratuite (512 Mo, mise en veille après 15 min d'inactivité). Aucun secret n'est nécessaire.

Déploiement Render (région Frankfurt, redéployé à chaque push sur master)

Champ	Valeur
Runtime	Python 3
Branch	master
Build Command	pip install -r requirements.txt
Start Command	streamlit run app.py --server.port \$PORT --server.address 0.0.0.0 --server.headless true
Health Check Path	/_stcore/health
PYTHON_VERSION	3.12.3

11. Déclaration IA et outils

- Dans le produit : un modèle tabulaire scikit-learn, sans LLM ni API d'IA externe. NVIDIA Brev : non utilisé.
- Pour développer : Claude Code (Anthropic) comme assistant de programmation — Claude Sonnet 4.6 pour le premier commit, Claude Opus 5.5 ensuite ; une session Claude séparée pour les 4 commits de finition de la pull request #1 ; des agents Claude en lecture seule pour l'audit de sécurité et une évaluation simulée selon la grille du jury. Skills : stockvisible-data-contract et dataviz. Aucun serveur MCP.
- Le porteur du projet fixe les règles et prend les décisions tracées dans l'historique Git ; chaque commit porte la mention Co-Authored-By correspondante.
- Chronologie (heures UTC, historique Git) : M0 → M10 le 26/09/2026 de 16:03 à 23:29 ; M11 → M17 le 27/09/2026 de 00:56 à 13:11. Les règles sur le code préparé avant l'événement sont celles du règlement officiel.

12. Licences

- Données : FreshRetailNet-50K, Dingdong-Inc, CC BY 4.0.
- Dépendances : bibliothèques open source listées dans requirements.txt (pandas, NumPy, scikit-learn, Streamlit, Plotly, pyarrow, pytest, ruff, pip-audit) et package.json (@playwright/test, outil de test uniquement), chacune sous sa propre licence.
- Code de StockVisible : aucun fichier de licence n'a encore été ajouté au dépôt ; sans licence explicite, les droits restent réservés à l'auteur.
- Logo de l'onglet : icône de Galsen Technologie (galsentechnologie.com).

13. Après le hackathon

- Pilote avec un point de vente équipé d'une caisse et d'un suivi de stock, avec ses coûts réels.
- Puis un moteur orienté achat (quantile supérieur à la médiane) choisi sur la validation et évalué sur la période « eval » officielle du jeu de données, jamais utilisée pour évaluer un modèle.
- Choisir une licence pour le code : aucun fichier de licence n'a encore été ajouté au dépôt.

Chronologie

L'historique Git fait foi. Premier commit le 26/09/2026 à 16:03 UTC ; données, modèle, gel, test final et écrans (M0 à M11) commités le 26/09/2026 et dans la nuit jusqu'au 27/09 à 00:56 UTC ; tests E2E, audit de sécurité, export, CI, production et

finitions le 27/09/2026.

14. Démo et audit jury

- Script et vidéo de démonstration (M18) : à venir.
- Audit jury versionné (M19) : à venir.